

ML Specification Report — playground-series-s4e11

Depression Prediction from Lifestyle Survey Data • AIDE agent (20-step tree search, Claude Sonnet via shim)

Metrics, step counts, timings and every leaderboard figure are taken verbatim from the run's `facts_aide.json` (derived from `journal.json` for run `2-illustrious-smooth-roadrunner`). Pipeline description comes from AIDE's own auto-generated `report.md` and from `best_solution.py`; task metadata from `config.yaml`. Figures that exist only in a node's plan/analysis text are attributed inline.

Contents	
Depression Prediction from Lifestyle Survey Data • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
Search trajectory	4
Specification fields	5
Inference procedures	6
Evaluation & Benchmarking	6

Overview

The task is binary classification of `Depression` from a demographic and lifestyle survey (metric: Accuracy, maximize). It is solved by **AIDE**, a tree-search coding agent that writes a complete solution script each step, executes it, reads back the printed CV score or the traceback, and mutates. Given a **20-step budget**, AIDE's champion appeared at **step 18** with **5-fold CV accuracy 0.94055** — a single CatBoost classifier on native categoricals, with the student/professional field merge, a parsed numeric `Sleep_Hours`, and fold-safe out-of-fold target encoding of the four highest-cardinality columns.

That submission scored **Public LB 0.94184 / Private LB 0.94056**, placing **1020 / 2687 — the 62.1st percentile**. The three-way private-leaderboard outcome was recorded as `local_winner = aide`, `lb_winner = nvidia`: AIDE had the best local number of the three lanes but finished **second of three on the private split**, behind NVIDIA (0.94076) and ahead of our from-scratch agent's then-current 0.94043. **The current benchmark record moves AIDE to third**: after that lane's final-architecture re-run,

`benchmark_results/rerun_three_way.csv` gives it private 0.94107 and records `winner_priv = mine`. The margins are tiny in absolute terms and are set out numerically in the Benchmarking section.

Why it matters. Depression screening from self-reported lifestyle and demographic survey items is the cheap end of mental-health triage — no clinician time, no instrument beyond a questionnaire. What makes this dataset interesting methodologically is that its missingness is structural rather than random: students and working professionals answer disjoint blocks of questions, so a naive imputer manufactures signal where none exists. The competition rewards handling that structure honestly, and the winning move here — merging the mutually exclusive pressure/satisfaction fields into shared columns — is exactly a fix for it.

Data

Purpose of Data. Classify each respondent as depressed or not — a binary problem scored on hard-label accuracy, not on ranking. **Data Format** is **tabular CSV** mixing numeric ratings with free-form-ish string categoricals. **Data Volume:** the competition config records **140,700 train rows / 93,800 test rows** with **18 features** excluding `id` and the target, and a **positive rate of 18.2%** — an imbalanced target scored on an accuracy metric, which is why the majority-class baseline already sits high and why the entire search lives inside a four-thousandths band.

Data Quality has two defining wrinkles. First, the **structured missingness:** per the config, `Academic Pressure / CGPA / Study Satisfaction` are ~80% null for working professionals while `Work Pressure / Job Satisfaction` are ~20% null for students — the two populations answer different questionnaires. Second, the **dirty high-cardinality strings:** AIDE's own report records typos, inconsistent casing and whitespace across `City`, `Degree`, `Profession`, `Sleep Duration` and `Dietary Habits` (its step-14 plan cites "Bagalore" versus "Bangalore" as the motivating example). Numeric NaNs were either median-imputed or left in place for learners with native support; categorical NaNs were cast to an explicit `"Missing"` string, since CatBoost requires strings rather than NaN in categorical columns.

Annotation Guidelines. The label is `Depression` $\in \{0, 1\}$, and the sample submission expects **hard 0/1 labels**, not probabilities. Every candidate must therefore commit to a decision rule. AIDE tested a per-fold accuracy-maximizing threshold at step 2 (0.93941) and its report notes the optimal cut averaged ≈ 0.498 — statistically indistinguishable from 0.5 — so the champion simply thresholds the fold-averaged probability at 0.5.

Feature Set. The champion's matrix is the 18 raw columns plus three engineered families:

Group	Features
Categorical, passed natively to CatBoost (10)	<code>Gender</code> , <code>City</code> , <code>Working Professional or Student</code> , <code>Profession</code> , <code>Sleep Duration</code> , <code>Dietary Habits</code> , <code>Degree</code> , <code>Have you ever had suicidal thoughts ?</code> , <code>Family History of Mental Illness</code> , <code>Name</code>
Numeric (raw)	

Group	Features
	the remaining survey ratings, including Academic Pressure , Work Pressure , Study Satisfaction , Job Satisfaction , CGPA , Financial Stress , Work/Study Hours , Age
Engineered — block merge	Pressure = Academic Pressure + Work Pressure (NaN → 0); Satisfaction = Study Satisfaction + Job Satisfaction (NaN → 0)
Engineered — parsed	Sleep_Hours , a numeric midpoint parsed from the Sleep Duration string ("7-8 hours" → 7.5)
Engineered — target encoding	City_te , Profession_te , Degree_te , Name_te — smoothed OOF means, prior weight 20

The merge exploits the disjointness directly: because a respondent answers one block or the other, a plain sum with NaN → 0 recovers a single well-populated "pressure" and "satisfaction" axis without inventing values. Note the target encodings are **added alongside** the raw string columns rather than replacing them — the model sees both CatBoost's ordered target statistics and a separate globally smoothed numeric summary. When AIDE tried the opposite at step 5, replacing native categorical handling with K-fold target encoding wholesale, it produced the worst score of the entire run (0.93651).

Splitting strategy. A single **StratifiedKFold(n_splits=5, shuffle=True, random_state=42)**, stratified because the 18.2% positive rate makes fold-wise class balance worth guaranteeing. The leak-free discipline that matters here is **fold-safe target encoding**, and the champion implements it correctly: the same `skf` object generates both the encoding folds and the model folds, so every encoded value for a validation row is computed from that row's complementary training fold only (`TE-fold == model-fold`); the test-side encoding is the average of the five fold mappings, and unseen levels fall back to the global mean. The encoding is smoothed toward the prior with weight 20. **Leaderboard status: submitted and scored** — Public 0.94184, Private 0.94056 — so this run's CV can be checked against a real held-out split, and it holds up remarkably well.

Models & Architecture

Purpose of Architecture. A binary depression classifier maximizing hard-label accuracy. **Architecture Type** is a **single CatBoost gradient-boosting classifier**, fold-bagged — not an ensemble of architectures. This is a notable outcome: AIDE evaluated LightGBM, XGBoost, HistGradientBoostingClassifier and a PyTorch MLP with categorical embeddings, then repeatedly tried to blend and stack them, and **every blend and every stack lost to standalone CatBoost**. Its report attributes this to LightGBM's errors being largely a subset of CatBoost's rather than complementary — no diversity to harvest.

Input Format is a mixed frame with the ten categorical columns handed to CatBoost as string `cat_features` indices via `Pool` , and all engineered columns as plain numerics. **Input Dimension** is the

18 raw features plus 2 merged, 1 parsed and 4 target-encoded columns per row — a flat vector, no sequence or spatial structure.

Architecture Description. The learner is `CatBoostClassifier(iterations=3000, learning_rate=0.05, depth=6, loss_function="Logloss", eval_metric="Accuracy", l2_leaf_reg=3.0, random_seed=42, early_stopping_rounds=100)` with `use_best_model=True`, fit once per fold. **Model Complexity** is best expressed as trees $\times$ leaves: five fold-models, each capped at 3000 symmetric (oblivious) depth-6 trees — 64 leaves apiece — with the effective count set by 100-round early stopping on the fold's validation accuracy; the realized iteration counts are **not recorded**. Note the deliberate split between the optimization loss (`Logloss` , smooth and differentiable) and the selection metric (`Accuracy` , the competition's), which is the right way to early-stop on a non-differentiable target.

Training procedures

There is no hand-designed stage ladder. The training procedure is the search: 20 sequential nodes, each a full rewrite-execute-analyse cycle against the same 5-fold protocol.

Search trajectory

Of the 20 steps, **19 scored and 1 was buggy**, with `buggy_exc_types` recording exactly one `CatBoostError` . Total execution was **1133.8 s** (mean **56.7 s**/node, max **180.2 s**) — comfortably inside the per-node budget; nothing timed out.

Step	CV Accuracy	Node (per AIDE's report)
0	0.93946	LightGBM, native categoricals, Pressure/Satisfaction merge
1	0.93909	XGBoost, frequency + label encoding, rare grouping
2	0.93941	LightGBM + per-fold tuned threshold (≈ 0.498)
3	0.94041	CatBoost , native categoricals
4	0.94005	CatBoost + LightGBM 50/50 blend
5	0.93651	LightGBM + K-fold TE replacing native categoricals
6	0.93877	PyTorch MLP with categorical embeddings
7	0.93891	HistGradientBoostingClassifier
8	0.93999	CatBoost + LightGBM 60/40 + Name frequency
9	0.9402	CatBoost + rare-category grouping
10	0.94006	CatBoost multi-seed bagging (3 seeds/fold)
11	0.94026	CatBoost + LightGBM stacking, logistic meta-model
12	0.94031	CatBoost, Financial Stress as categorical
13	0.93975	CatBoost, Lossguide growth + Age-Group bucket
14	buggy	CatBoost <code>text_features</code> on City/Profession/Degree → <code>CatBoostError</code>

Step	CV Accuracy	Node (per AIDE's report)
15	0.94041	same idea, text columns reverted to plain categoricals (bugfix)
16	0.94004	CatBoost + Ordered boosting
17	0.94038	CatBoost + Sleep_Hours numeric feature
18	0.94055	CatBoost + Sleep_Hours + OOF target encoding — champion
19	0.94026	champion + Financial Stress × Pressure interaction

The whole 20-step search bought this much:

```
0.94055 - 0.93946 = 0.00109    (first scored node → champion)
0.00109 / 0.93946 = 0.00116    → 0.12% relative accuracy gain
```

The plateau is the story here, and it should be stated plainly. CatBoost arrives at **step 3** and scores 0.94041 immediately. Every one of the sixteen subsequent nodes is either a CatBoost variant or a failed attempt to add a second architecture, and their combined yield is:

```
0.94055 - 0.94041 = 0.00014    (step 3 → step 18, across 15 further steps)
```

Fourteen-hundred-thousandths of accuracy for three-quarters of the budget. Excluding the single collapse at step 5, the entire scored trajectory lives inside a band of

```
0.94055 - 0.93651 = 0.00404    (full range, best to worst)
```

which is consistent with AIDE's own conclusion that the dataset has a hard accuracy ceiling near 94%. The search behaved sensibly — it did not thrash, and it did make monotone-ish progress in the final third (0.94038 → 0.94055) — but it never found a structurally different idea after step 3, and its two genuinely creative swings both failed: the MLP (0.93877) and the CatBoost text-feature experiment (crash).

The one failure was cheap and correctly diagnosed. At step 14 AIDE hypothesized that treating the misspelled string columns as CatBoost text features would let n-gram statistics capture partial string similarity that discrete categorical statistics cannot. CatBoost raised `Dictionary size is 0` while building the text estimators — the columns are short labels, not natural language, so the tokenizer had nothing to build a dictionary from. The exception was raised before training began, so the node cost almost no wall-clock, and the very next step reverted the columns to plain categoricals and recovered 0.94041 exactly. That is the tree-search loop working as intended: an error message read, diagnosed, and repaired in one hop.

Specification fields

The **Loss Function** is **Logloss** (CatBoost's binary cross-entropy), with **model selection and early stopping on Accuracy** — deliberately decoupled, since the competition metric is non-differentiable. The **Optimization Algorithm** is gradient boosting with ordered target statistics and symmetric

(oblivious) trees — **not** SGD or ADAM. The **Learning Rate** is the boosting shrinkage, **0.05**. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by early stopping on fold accuracy, not a schedule) and **no Batch Size** (N/A — full-dataset boosting, not mini-batched).

Training Duration across the whole search was **1133.8 s** (mean **56.7 s**/node, max **180.2 s**); per the trajectory, the expensive nodes were the multi-seed and stacking variants, not the champion. **Training Memory** is **not recorded**. **Transfer Learning** is N/A (no pretrained weights); the AIDE analogue is **within-run journal memory** — step 18's plan opens by naming "the best standalone CatBoost run (0.94041)" and proposes a single atomic addition to it, which is how the agent carries state. That memory does not persist across competitions; AIDE begins each episode from the raw task description. **Data Augmentation** is N/A (tabular); the analogues are feature engineering (the block merge, sleep parsing) and target encoding, plus the multi-seed bagging tried at step 10 (0.94006 — no gain).

Reproducibility Standards are moderate. `random_seed=42` is fixed in CatBoost and `random_state=42` in the fold splitter, and the encoding is deterministic, so `best_solution.py` is re-runnable end to end. There is **no digit-for-digit rebuild gate** in the AIDE lane — the artifact of record is the emitted script, not a verified reconstruction. One structural caveat: the champion configuration was chosen by comparing 19 candidates on the same 5-fold OOF partition, so 0.94055 is **selection-optimistic by construction**. Unusually, the leaderboard says that optimism was negligible here (see below).

Inference procedures

Decision Threshold is **0.5**, applied to the mean of the five fold-models' predicted probabilities. This is a real choice, not a default: step 2 measured a per-fold accuracy-optimizing threshold and found it averaged ≈ 0.498 with no meaningful accuracy gain, so the champion keeps the natural cut. **Post-processing** beyond thresholding is none — no calibration, no rank transform. Test predictions are produced by averaging `predict_proba` across the five fold-models (`test_preds_proba += proba / 5`) and then hardened to 0/1, matching the sample submission's label format. **Inference Duration** and **Inference Memory** are **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Classify each held-out respondent as depressed or not.

Performance Metrics. Accuracy (maximize) is the sole metric. AIDE's champion reaches **CV accuracy 0.94055**, improving from a first scored node of 0.93946 — the 0.00109 / 0.12% gain computed above. What is genuinely impressive is the **CV-to-private agreement**:

$0.94056 - 0.94055 = 0.00001$	(private LB - local CV)
$0.94184 - 0.94056 = 0.00128$	(public LB - private LB)

A one-hundred-thousandth gap between a 5-fold OOF estimate and the private leaderboard is about as tight as this kind of validation gets, and it retroactively vindicates the fold-safe encoding discipline: had

the target encoding leaked, the CV would have run optimistically high and the private score would have come in below it. The public split is the outlier, sitting 0.00128 above private — a reminder that public-LB movement at this scale is noise, not signal.

Performance Benchmarking. All three agent lanes submitted to the same private split:

Agent	Approach	Private LB Accuracy
NVIDIA	reproduce-agent lane	0.94076
AIDE	20-step tree search, single CatBoost + fold-safe TE	0.94056
Our from-scratch agent	GBDT pool + tree search	0.94043

$0.94076 - 0.94056 = 0.00020$	(NVIDIA over AIDE)
$0.94056 - 0.94043 = 0.00013$	(AIDE over our agent)

The recorded verdict splits: `local_winner = aide`, `lb_winner = nvidia`. AIDE posted the strongest local figure of the three lanes and still finished second of three on the private split, losing by 0.00020 — a margin at the fourth decimal place, i.e. a handful of flipped labels. Those are the development-era figures: on the current benchmark record the from-scratch lane's final-architecture re-run scores private 0.94107, which puts AIDE third of the three and awards this competition to that lane (`winner_priv = mine` in `benchmark_results/rerun_three_way.csv`). AIDE's submission ranks **1020 / 2687, the 62.1st percentile**. The fair reading is that all three agents converged onto the same 94%-accuracy plateau that AIDE's own report identifies, and the ordering between them at the fourth decimal place is not a meaningful capability difference — it is which side of a coin-flip each landed on. The one substantive observation is that AIDE reached that plateau with a simpler artifact than either competitor: one CatBoost, five folds, four engineered columns, and no blending at all.

Fields marked "not recorded": realized boosting-iteration counts after early stopping; training peak memory; inference duration and memory. The single buggy node's wall-clock cost is recorded in the journal but only the run-level totals (1133.8 s / 56.7 s mean / 180.2 s max) are carried in `facts_aide.json`.